UNIDAD 4 – DOM, eventos y formularios

DESARROLLO WEB EN ENTORNO CLIENTE

Índice

- Formularios.
- DOM.
- Eventos.

Formularios

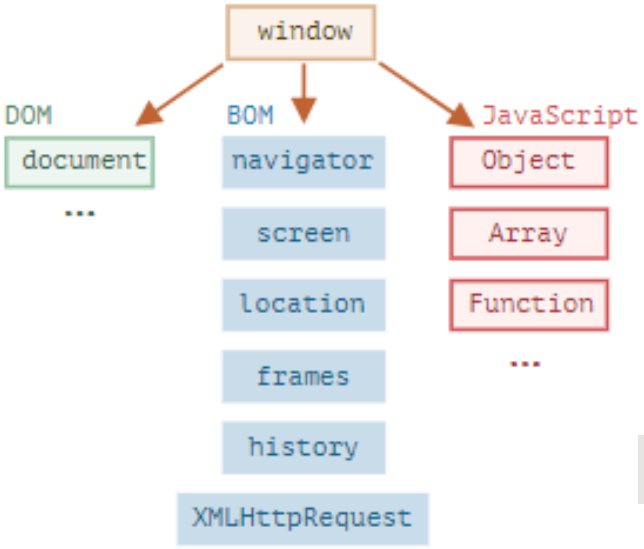
- ¿Qué es un formulario?

- Principales etiquetas de un formulario.
  - Form.
  - Input.
  - Textarea.
  - Checkbox
  - Radio.
  - Select.
  - Button.

- BootStrap.- conceptos básicos
  - ¿Qué es?
  - Como usarlo.
  - Clases principales.

Documento

- Entorno del navegador. Especificaciones.




```
1  function sayHi() {
2    alert("Hola");
3  }
4
5  // Las funciones globales son métodos del objeto global:
6  window.sayHi();
```

```
1  alert(window.innerHeight); // altura interior de la ventana
```

- Hay un objeto "raíz" llamado window. Tiene dos roles:
  - Objeto global para el código JavaScript.
  - Representa la "ventana del navegador" y proporciona métodos para controlarla.

Documento

- DOM (Modelo de Objetos del Documento).

  Representa todo el contenido de la página como objetos que pueden ser modificados.

  El objeto **document** es el punto de entrada a la página. Con él podemos cambiar o crear cualquier cosa en la página.

```
1  // cambiar el color de fondo a rojo
2  document.body.style.background = "red";
3
4  // deshacer el cambio después de 1 segundo
5  setTimeout(() => document.body.style.background = "", 1000);
```

Documento

- BOM (Modelo de Objetos del Navegador).

  El Modelo de Objetos del Navegador son objetos adicionales proporcionados por el navegador (entorno host) para trabajar con todo excepto el documento.

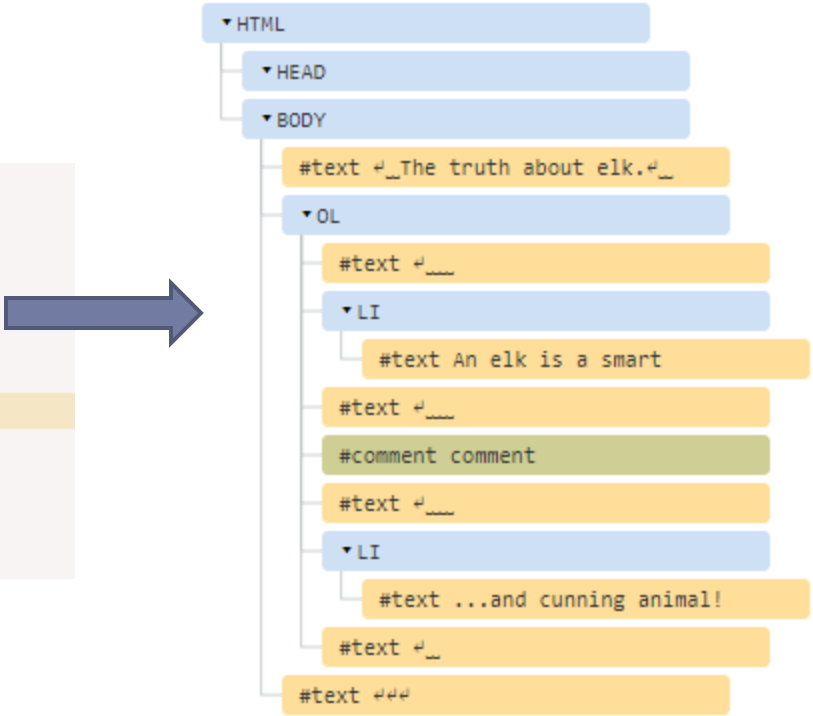
```
1  alert(location.href); // muestra la URL actual
2  if (confirm("Ir a wikipedia?")) {
3    location.href = "https://wikipedia.org"; // redirigir el navegador a otra URL
4  }
```

Documento

- Según el *Modelo de Objetos del Documento* (DOM), cada etiqueta HTML es un objeto. Las etiquetas anidadas son llamadas "hijas" de la etiqueta que las contiene. El texto dentro de una etiqueta también es un objeto.

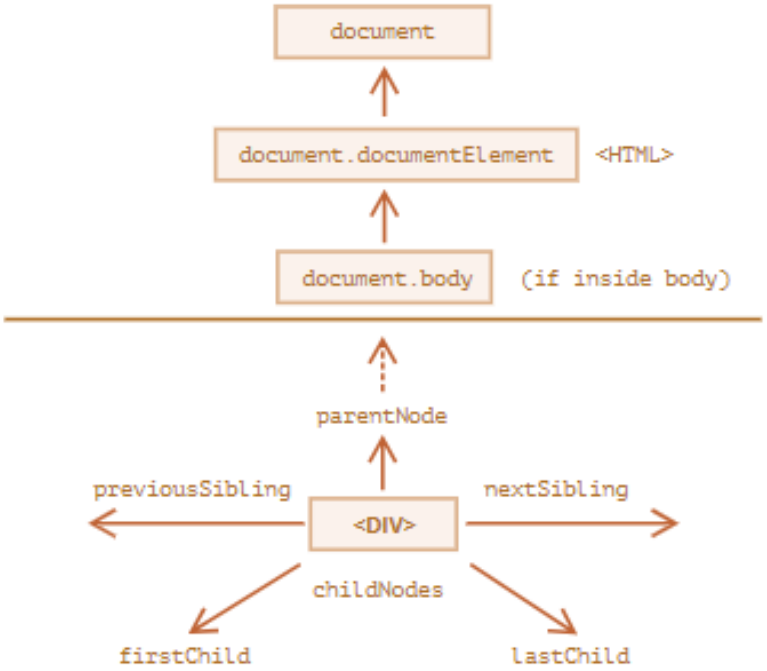
```
1  document.body.style.background = 'red'; // establece un color de fondo rojo
2
3  setTimeout(() => document.body.style.background = '', 3000); // volver atrás
```

```
1  <!DOCTYPE HTML>
2  <html>
3  <body>
4    The truth about elk.
5    <ol>
6      <li>An elk is a smart</li>
7      <!-- comentario -->
8      <li>...y el astuto animal!</li>
9    </ol>
10 </body>
11 </html>
```



Documento

- Recorriendo el DOM.



Documento

- Recorriendo el DOM. Hijos childNodes, firstChild, lastChild.

```html
<html>
<body>
  <div>Begin</div>

  <ul>
    <li>Information</li>
  </ul>

  <div>End</div>

  <script>
    for (let i = 0; i < document.body.childNodes.length; i++) {
      alert( document.body.childNodes[i] ); // Texto, DIV, Texto, UL, ..., SCRIPT
    }
  </script>
  ...más cosas...
</body>
</html>
```

```
elem.childNodes[0] === elem.firstChild
elem.childNodes[elem.childNodes.length - 1] === elem.lastChild
```

Documento

- Hermanos y el padre.

```
// el padre de <body> es <html>
alert( document.body.parentNode === document.documentElement ); // verdadero

// después de <head> va <body>
alert( document.head.nextSibling ); // HTMLBodyElement

// antes de <body> va <head>
alert( document.body.previousSibling ); // HTMLHeadElement
```

Documento

- Crear nodos DOM.
  - Métodos appendChild y createElement.

```
// 1. Crear el elemento <p> (párrafo)
const nuevoParrafo = document.createElement('p');

// 2. Añadir texto y una clase CSS
nuevoParrafo.textContent = '¡Hola, este es un texto creado con JavaScript!';
nuevoParrafo.className = 'mi-clase';

// 3. Seleccionar el contenedor padre en el DOM
const contenedor = document.getElementById('contenedor');

// 4. Insertar el nuevo elemento dentro del contenedor
contenedor.appendChild(nuevoParrafo);
```

Documento

- Algunos tipos de elementos del DOM, por ejemplo las tablas, proveen propiedades adicionales y colecciones para acceder a su contenido.

```
<table id="table">
  <tr>
    <td>one</td><td>two</td>
  </tr>
  <tr>
    <td>three</td><td>four</td>
  </tr>
</table>

<script>
  // seleccionar td con "dos" (primera fila, segunda columna)
  let td = table.rows[0].cells[1];
  td.style.backgroundColor = "red"; // destacarlo
</script>
```

Documento

- **Buscar: getElement*, querySelector***

```html
<div id="elem">
  <div id="elem-content">Elemento</div>
</div>

<script>
  // obtener el elemento
  let elem = document.getElementById('elem');

  // hacer que su fondo sea rojo
  elem.style.background = 'red';
</script>
```

```html
<ul>
  <li>La</li>
  <li>prueba</li>
</ul>
<ul>
  <li>ha</li>
  <li>pasado/li>
</ul>
<script>
  let elements = document.querySelectorAll('ul > li:last-child');

  for (let elem of elements) {
    alert(elem.innerHTML); // "prueba", "pasado"
  }
</script>
```

Documento

- getElementsBy*.
  - getElementById(id).
  - getElementsByTagName(tag).
  - getElementsByClassName(className).
  - getElementsByName(name).

```
<table id="table">
  <tr>
    <td>Su edad:</td>

    <td>
      <label>
        <input type="radio" name="age" value="young" checked> menos de 18
      </label>
      <label>
        <input type="radio" name="age" value="mature"> de 18 a 50
      </label>
      <label>
        <input type="radio" name="age" value="senior"> más de 60
      </label>
    </td>
  </tr>
</table>

<script>
  let inputs = table.getElementsByTagName('input');

  for (let input of inputs) {
    alert( input.value + ': ' + input.checked );
  }
</script>
```

Documento

- Colecciones vivas.
  - getElementsBy*: *live collection*
  - querySelectorAll: *static collection*.

```
1  <div>Primer div</div>
2
3  <script>
4    let divs = document.getElementsByTagName('div');
5    alert(divs.length); // 1
6  </script>
7
8  <div>Segundo div</div>
9
10 <script>
11   alert(divs.length); // 2
12 </script>
```

```
1  <div>Primer div</div>
2
3  <script>
4    let divs = document.querySelectorAll('div');
5    alert(divs.length); // 1
6  </script>
7
8  <div>Segundo div</div>
9
10 <script>
11   alert(divs.length); // 1
12 </script>
```

Documento

- getElementById - querySelector.
  - Son de los métodos de para seleccionar/buscar elementos en el DOM que usaremos mas asiduamente.

```
element = document.querySelector(selectores);
```

```
var el = document.querySelector(".miClase");
```

Ejemplo más útil

Los *Selectores* pueden ser muy útiles como se demostrará en el siguiente ejemplo. Aquí, será retornado el primer elemento `<input name="login" />` dentro de `<div class="user-panel main">`.

```
var el = document.querySelector("div.user-panel.main
input[name='login']");
```

Documento

- Contenido: innerHTML.

```
<body>
  <p>Un párrafo</p>
  <div>Un div</div>

  <script>
    alert( document.body.innerHTML ); // leer el contenido actual
    document.body.innerHTML = 'El nuevo BODY!'; // reemplazar
  </script>

</body>
```

- HTML completo del elemento: outerHTML.

```
<div id="elem">Hola <b>Mundo</b></div>

<script>
  alert(elem.outerHTML); // <div id="elem">Hola <b>Mundo</b></div>
</script>
```

Documento.

- Propiedad hidden.

```
<div>Ambos divs a continuación están ocultos</div>

<div hidden>Con el atributo "hidden"</div>

<div id="elem">JavaScript asignó la propiedad "hidden"</div>

<script>
  elem.hidden = true;
</script>
```

```
<div id="elem">Un elemento parpadeante</div>

<script>
  setInterval(() => elem.hidden = !elem.hidden, 1000);
</script>
```

- Mas propiedades.

```
<input type="text" id="elem" value="value">

<script>
  alert(elem.type); // "text"
  alert(elem.id); // "elem"
  alert(elem.value); // value
</script>
```

Documento

- Atributos y propiedades.

Los métodos para trabajar con atributos son:

- `elem.hasAttribute(nombre)` – para comprobar si existe.
- `elem.getAttribute(nombre)` – para obtener el valor.
- `elem.setAttribute(nombre, valor)` – para dar un valor.
- `elem.removeAttribute(nombre)` – para eliminar el atributo.
- `elem.attributes` es una colección de todos los atributos.

- **Todos los atributos que comienzan con "data-" están reservados para el uso de los programadores. Están disponibles en la propiedad dataset.**

```
// leyendo el valor
alert(elem.dataset.widgetName);
// o
alert(elem.getAttribute('data-widget-name'));
```

Documento

- Modificando el documento.

  Muestra un mensaje usando HTML/CSS.

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<div class="alert">
  <strong>¡Hola!</strong> Usted ha leído un importante mensaje.
</div>
```

¡Hola! Usted ha leído un importante mensaje.

Documento

- Modificando el documento. createElement() y createTextNode.

  Muestra un mensaje usando usando javascript usando método de inserción append().

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<script>
  let div = document.createElement('div');
  div.className = "alert";
  div.innerHTML = "<strong>¡Hola!</strong> Usted ha leído un importante mensaje.";

  document.body.append(div);
</script>
```

Documento

- ## Modificando el documento.

Aquí hay más métodos de inserción, ellos especifican diferentes lugares donde insertar:

- `node.append(...nodos o strings)` – agrega nodos o strings *al final* de `node`,
- `node.prepend(...nodos o strings)` – insert nodos o strings *al principio* de `node`,
- `node.before(...nodos o strings)` –- inserta nodos o strings *antes* de `node`,
- `node.after(...nodos o strings)` –- inserta nodos o strings *después* de `node`,
- `node.replaceWith(...nodos o strings)` –- reemplaza `node` con los nodos o strings dados.

```html
<ol id="ol">
  <li>0</li>
  <li>1</li>
  <li>2</li>
</ol>

<script>
  ol.before('before'); // inserta el string "before" antes de <ol>
  ol.after('after'); // inserta el string "after" después de <ol>

  let liFirst = document.createElement('li');
  liFirst.innerHTML = 'prepend';
  ol.prepend(liFirst); // inserta liFirst al principio de <ol>

  let liLast = document.createElement('li');
  liLast.innerHTML = 'append';
  ol.append(liLast); // inserta liLast al final de <ol>
</script>
```

Documento

- Modificando el documento.

Pero ¿y si queremos insertar un string HTML "como html", con todas las etiquetas y demás funcionando, de la misma manera que lo hace `elem.innerHTML`?

insertAdjacentHTML/Text/Element

Para ello podemos usar otro métodos, muy versátil: `elem.insertAdjacentHTML(where, html)`.

El primer parámetro es un palabra código que especifica dónde insertar relativo a `elem`. Debe ser uno de los siguientes:

- `"beforebegin"` – inserta `html` inmediatamente antes de `elem`
- `"afterbegin"` – inserta `html` en `elem`, al principio
- `"beforeend"` – inserta `html` en `elem`, al final
- `"afterend"` – inserta `html` inmediatamente después de `elem`

El segundo parámetro es un string HTML, que es insertado "como HTML".

```
<div id="div"></div>
<script>
  div.insertAdjacentHTML('beforebegin', '<p>Hola</p>');
  div.insertAdjacentHTML('afterend', '<p>Adiós</p>');
</script>
```

Documento

- Modificando el documento.

  Eliminado nodos. Node.remove().

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<script>
  let div = document.createElement('div');
  div.className = "alert";
  div.innerHTML = "<strong>¡Hola!</strong> Usted ha leído un importante me

  document.body.append(div);
  setTimeout(() => div.remove(), 1000);
</script>
```

Documento

- Modificando el documento.

  Clonando nodos. Node.cloneNode(boolean).

  Si parámetro = true → Clonación profunda.

  Si parámetro = false → Clonación sin hijos.

```
<style>
.alert {
  padding: 15px;
  border: 1px solid #d6e9c6;
  border-radius: 4px;
  color: #3c763d;
  background-color: #dff0d8;
}
</style>

<div class="alert" id="div">
  <strong>¡Hola!</strong> Usted ha leído un importante mensaje.
</div>

<script>
  let div2 = div.cloneNode(true); // clona el mensaje
  div2.querySelector('strong').innerHTML = '¡Adiós!'; // altera el clon

  div.after(div2); // muestra el clon después del div existente
</script>
```

Documento

- Estilos.

  La propiedad elem.style es un objeto que corresponde a lo escrito en el atributo "style". Establecer elem.style.width="100px" funciona igual que sí tuviéramos en el atributo style una cadena con width:100px.

```
// si ejecutamos este código, el <body> parpadeará
document.body.style.display = "none"; // ocultar

setTimeout(() => document.body.style.display = "", 1000); // volverá
```

```
document.body.style.background = 'red'; //establece background a rojo

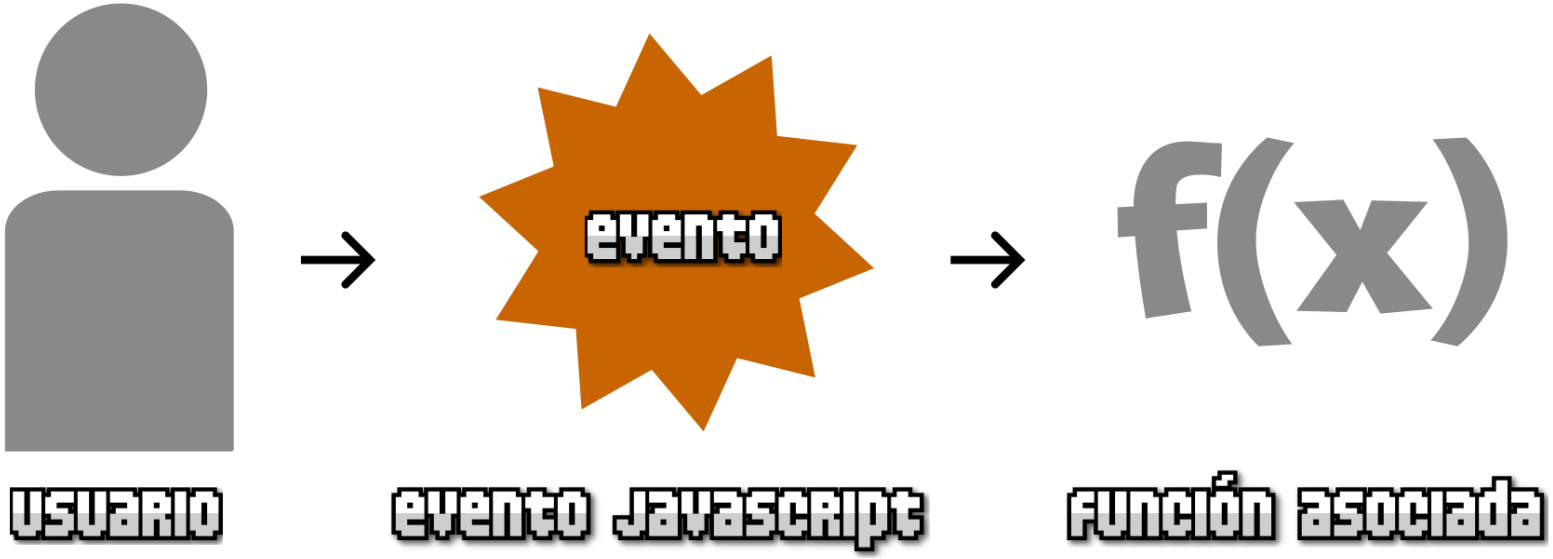
setTimeout(() => document.body.style.removeProperty('background'), 1000);
```

```
<div id="div">Button</div>

<script>
  // podemos establecer estilos especiales con banderas como "important"
  div.style.cssText=`color: red !important;
    background-color: yellow;
    width: 100px;
    text-align: center;
  `;

  alert(div.style.cssText);
</script>
```

Eventos



Eventos

- *Un evento* es una señal de que algo ocurrió. Todos los nodos del DOM generan dichas señales (pero los eventos no están limitados sólo al DOM).

  Aquí hay una lista con los eventos del DOM más utilizados, solo para echar un vistazo:

Eventos del mouse:

- `click` – cuando el mouse hace click sobre un elemento (los dispositivos touch lo generan con un toque).
- `contextmenu` – cuando el mouse hace click derecho sobre un elemento.
- `mouseover` / `mouseout` – cuando el cursor del mouse ingresa/abandona un elemento.
- `mousedown` / `mouseup` – cuando el botón del mouse es presionado/soltado sobre un elemento.
- `mousemove` – cuando el mouse se mueve.

Eventos del teclado:

- `keydown` / `keyup` – cuando se presiona/suelta una tecla.

Eventos del elemento form:

- `submit` – cuando el visitante envía un `<form>`.
- `focus` – cuando el visitante hace foco en un elemento, por ejemplo un `<input>`.

Eventos del documento:

- `DOMContentLoaded` --cuando el HTML es cargado y procesado, el DOM está completamente construido

Eventos del CSS:

- `transitionend` – cuando una animación CSS concluye.

Eventos

- **Controladores de eventos.**

  Para reaccionar a los eventos podemos asignar un *handler (controlador)* el cual es una función que se ejecuta en caso de un evento.

  Los handlers son una forma de ejecutar código JavaScript en caso de acciones por parte del usuario.

```
<script>
  function countRabbits() {
    for(let i=1; i<=3; i++) {
      alert("Conejo número " + i);
    }
  }
</script>

<input type="button" onclick="countRabbits()" value="¡Cuenta los conejos!">
```

- Como propiedad del DOM.

```
<input id="elem" type="button" value="Haz click en mí">
<script>
  elem.onclick = function() {
    alert('¡Gracias!');
  };
</script>
```

Eventos

- **Controladores de eventos.**

  Como solo hay una propiedad onclick, no podemos asignar más de un handler.

  En el siguiente ejemplo se agrega un handler con JavaScript que sobrescribe el handler existente:

```
<input type="button" id="elem" onclick="alert('Antes')" value="¡Haz click en m
<script>
  elem.onclick = function() { // sobrescribe el handler existente
    alert('Después'); // solo se mostrará este
  };
</script>
```

- Para solucionar esto debemos usar addEventListener.

```
<input id="elem" type="button" value="Haz click en mí"/>

<script>
  function handler1() {
    alert('¡Gracias!');
  };

  function handler2() {
    alert('¡Gracias de nuevo!');
  }

  elem.onclick = () => alert("Hola");
  elem.addEventListener("click", handler1); // Gracias!
  elem.addEventListener("click", handler2); // Gracias de nuevo!
</script>
```

Eventos

- Delegación de eventos.

  La idea básica es que si tenemos muchos elementos manejados de manera similar podemos, en lugar de asignar un manejador a cada uno de ellos, poner un único manejador común.

Para manejadores de nivel de documento: siempre `addEventListener`

Cuando asignamos un manejador de evento al objeto `document`, debemos usar siempre `addEventListener`, no `document.on<event>`, porque este último causa conflictos: los manejadores nuevos sobrescribirán los viejos.

En proyectos reales es normal que haya muchos manejadores en `document`, asignados en diferentes partes del código.

Eventos

- Delegación de eventos. EJEMPLO Calculadora.

```
<script>
 const teclado = document.getElementsByClassName("calculator-keys")[0];

 teclado.addEventListener("click", manejadorEvento);

 function manejadorEvento(event) {
     if (event.target.tagName == "BUTTON") {
      const salida = document.getElementById("pantalla");
      let digito = event.target.value;
      salida.value += digito;
    }
  }

</script>
```

Eventos

- Acciones predeterminadas del navegador.

  Muchos eventos conducen automáticamente a determinadas acciones realizadas por el navegador.

  Por ejemplo:

  - Un clic en un enlace: inicia la navegación a su URL.
  - Un clic en el botón de envío de un formulario inicia su envío al servidor.
  - Al presionar un botón del ratón sobre un texto y moverlo, se selecciona el texto.
- Evitar las acciones del navegador.
  - La forma principal es utilizar el objeto event. Hay un método **event.preventDefault().**

Eventos

- Evitar las acciones del navegador. Ejemplo.

```
<p>Haz clic derecho aquí para el menú contextual del documento</p>
<button id="elem">Haz clic derecho aquí para el menú contextual del botón</button>

<script>
document.getElementById("elem").addEventListener("contextmenu", menu1);
document.addEventListener("contextmenu", menu2);

function menu1(event){
    event.preventDefault();
    alert("Menú contextual del botón");
}

 function menu2(event){
    event.preventDefault();
    alert("Menú contextual del documento");
}
</script>
```

- El problema es que cuando hacemos clic en elem, obtenemos dos menús: el de nivel de botón y (el evento emerge) el menú de nivel de documento.
- ¿Como arreglarlo? Una de las soluciones es pensar así: "Cuando hagamos clic con el botón derecho en el controlador de botones, detengamos su propagación" y usemos event.stopPropagation():

Aclaraciones



Aclaraciones

- Si quieres que el HTML se cargue primero, pero el JS se ejecute en cuanto el DOM esté listo:

```html
<head>
  <script defer src="mi-script.js"></script>
</head>
```

- Uso de ficheros .js en el head con DOM.

  La carga del DOM es posterior a la traspilación del fichero .js.

```
document.addEventListener("DOMContentLoaded", function () {
        document.body.style.backgroundColor = "red";
 });

o

document.addEventListener("DOMContentLoaded", () => document.body.style.backgroundColor = "red");
```

- Se recomienda usar la "ruta" completa de métodos de objetos DOM en lugar de su forma corta (usar directamente el id del elemento).
- Usar variables javascript para almacenar la "ruta" completa de métodos de objetos DOM.

```
let boton1 = document.getElementById("b1");

boton1.addEventListener("click", func);



function func(){
  alert(boton1.getAttribute("data-fila"));
}
```